HirQube

AI-Powered Global Job Platform

COMPLETE BUILD GUIDE

Day-by-Day Blueprint: From Setup to Production

Version 2.0 | March 2026

30 Days	4 Phases	1M+
Total Duration	Development Phases	Jobs Target

Tech Stack: FastAPI • Firebase • React • Flutter • Python AI • AWS

PROJECT OVERVIEW & MASTER ARCHITECTURE

What You Are Building

HirQube is a next-generation AI-powered global job, internship, and hiring platform. It goes beyond LinkedIn, Indeed, and Glassdoor by combining real-time job aggregation from 8+ sources, an AI-driven ATS scoring engine, a Neural Interview System, intelligent resume parser, skill gap analyzer, and a full recruiter verification system — all on one platform.

Current State of Your Codebase (Analyzed)

> Your HirQube-main project has: Node.js/Express backend, Firebase Firestore DB, 7 job source services (Adzuna, RemoteOK, GitHub, StackOverflow, Dev.to, LinkedIn scraper, Indeed via Adzuna), a React frontend with job cards and detail modal, and a daily cron scheduler. The Job.js model is still in Mongoose format — needs migration to Python/FastAPI.

What You Have ✅	What Needs Building 🔨
Express + Firebase backend	FastAPI Python backend (AI-ready)
7 job source integrations	3 more sources (Jooble, Internshala, Reed)
Daily cron job scheduler	Airflow distributed pipeline
React job cards UI	Full jobseeker/recruiter dashboards
Company logo fetching	Company intelligence (funding, Glassdoor, HR profiles)
Fingerprint deduplication	ATS scoring engine + Resume parser
Basic job search	AI recommendations + skill gap analysis
	Neural Interview Engine
	Flutter mobile app
	Full authentication (OTP, Google, OAuth)

Full System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        HIRQUBE PLATFORM                              │
├──────────────────┬──────────────────┬──────────────────┬──────────────────┤
│  React Web       │  Flutter App     │  Admin Panel     │  Recruiter Portal │
│  (Frontend)      │  (Mobile)        │  (React)         │  (React)         │
├──────────────────┴──────────────────┴──────────────────┴──────────────────┤
│                      API GATEWAY (FastAPI)                              │
├──────────────────┬──────────────────┬──────────────────┬──────────────────┤
│  Auth API        │  Jobs API        │  AI API          │  Interview API   │
│  /auth/*         │  /jobs/*         │  /ai/*           │  /interview/*    │
├──────────────────┴──────────────────┴──────────────────┴──────────────────┤
│                      AI LAYER (Python)                                  │
│  SpaCy NLP  │  Sentence-Transformers  │  PyTorch  │  Claude API        │
├──────────────────────────────────┬──────────────────────────────────────┤
│  DATA PIPELINE                   │  SCRAPING SYSTEM                     │
```

```
│  Airflow + Redis + Celery    │  Scrapy + rotating proxies         │
├──────────────────────────────┼────────────────────────────────────┤
│                     DATABASE LAYER                                 │
│  Firebase Firestore (main)   │  Redis Cache   │  Algolia Search   │
└────────────────────────────────────────────────────────────────────┘
```

Master 30-Day Schedule

Days	Phase / Focus	Key Deliverable	Result
Day 1–3	Foundation & Job APIs	FastAPI + 8 sources	**500+ jobs/day**
Day 4–6	Auth + DB Schema	Firebase 100+ collections	**Full auth flow**
Day 7–9	Company Intelligence	Logos, HR profiles, funding	**Rich job data**
Day 10–13	AI Core (ATS + Resume)	ATS 0-100, parser, gaps	**AI scoring live**
Day 14–17	Frontend Dashboards	Jobseeker + Recruiter UI	**Full UI ready**
Day 18–21	AI Interview Engine	Neural Interview + scoring	**AI interviews live**
Day 22–25	Flutter Mobile + Notifications	iOS/Android app	**Mobile ready**
Day 26–30	Scaling + AWS Deployment	Production ready	**LIVE** 🚀

DAY 1 — Foundation Setup & FastAPI Backend

Goal: Migrate from Node.js/Express to Python/FastAPI, set up project structure, configure Firebase, and get first API endpoint running

Task	Output
Environment setup + Python venv	Virtual env running
FastAPI project structure	Folder skeleton ready
Firebase Admin SDK config	DB connection live
First /health + /jobs endpoints	API returning data
Port Adzuna service to Python	500 jobs fetching

STEP 1: Install Python & Create Project

1.1 Install Required Tools

> Install Python 3.11+, pip, virtualenv, and Git if not already installed. On Windows use WSL2 (Ubuntu) for best compatibility.

```
# Windows (WSL2) or Ubuntu/macOS
sudo apt update && sudo apt install python3.11 python3-pip python3-venv git -y

# Verify installation
python3 --version   # Should show 3.11+
pip3 --version
```

1.2 Create Project Structure

```
# Create the full HirQube directory structure
mkdir -p hirqube/{backend,frontend,mobile,ai,scraper,infra}
cd hirqube/backend

# Create Python virtual environment
python3 -m venv venv
source venv/bin/activate        # Linux/Mac
# venv\Scripts\activate         # Windows
```

1.3 Project Folder Structure

```
hirqube/
├── backend/                    ← FastAPI Python backend
│   ├── app/
│   │   ├── main.py             ← Entry point
│   │   ├── config.py           ← Settings + env vars
│   │   ├── database.py         ← Firebase connection
│   │   ├── models/             ← Pydantic schemas
│   │   ├── routers/            ← API route handlers
│   │   ├── services/           ← Job API integrations
│   │   ├── ai/                 ← AI engine modules
│   │   └── scheduler/          ← Cron / Airflow tasks
│   ├── requirements.txt
```

```
│   └── .env
├── frontend/                    ← React webapp
├── mobile/                      ← Flutter app
└── scraper/                     ← Scrapy spiders
```

STEP 2: Install All Python Dependencies

2.1 Create requirements.txt

Save this as backend/requirements.txt — this is your complete dependency list for the entire project:

```
# Web Framework
fastapi==0.109.0
uvicorn[standard]==0.27.0
python-multipart==0.0.6

# Firebase
firebase-admin==6.4.0

# HTTP Requests
httpx==0.26.0
aiohttp==3.9.1
requests==2.31.0

# AI & NLP
spacy==3.7.2
transformers==4.37.0
sentence-transformers==2.3.1
torch==2.1.2
scikit-learn==1.4.0
numpy==1.26.3
pandas==2.1.4

# Resume Parsing
pypdf2==3.0.1
python-docx==1.1.0
pdfplumber==0.10.3

# Auth
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4

# Caching
redis==5.0.1

# Scheduler
apscheduler==3.10.4

# Utilities
python-dotenv==1.0.0
pydantic==2.5.3
pydantic-settings==2.1.0

# Install all
pip install -r requirements.txt

# Download SpaCy English model
python -m spacy download en_core_web_lg
```

STEP 3: Configure Firebase

3.1 Firebase Project Setup

1. Go to https://console.firebase.google.com
2. Click 'Create Project' → name it 'hirqube-prod'
3. Enable Firestore Database → Start in production mode
4. Go to Project Settings → Service Accounts
5. Click 'Generate new private key' → download JSON
6. Rename to 'firebase-service-account.json'
7. Place in backend/ folder (NEVER commit this to Git!)

3.2 Create .env File

```
# backend/.env

# Firebase
FIREBASE_SERVICE_ACCOUNT=./firebase-service-account.json
FIREBASE_PROJECT_ID=hirqube-prod

# Job APIs
ADZUNA_APP_ID=your_adzuna_app_id
ADZUNA_APP_KEY=your_adzuna_app_key
JOOBLE_API_KEY=your_jooble_key
RAPIDAPI_KEY=your_rapidapi_key

# AI
ANTHROPIC_API_KEY=your_claude_api_key

# Redis Cache
REDIS_URL=redis://localhost:6379

# Auth
SECRET_KEY=your-super-secret-jwt-key-change-this
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=1440
```

3.3 database.py — Firebase Connection

```
# backend/app/database.py
import firebase_admin
from firebase_admin import credentials, firestore
from app.config import settings

def init_firebase():
    if not firebase_admin._apps:
        cred = credentials.Certificate(settings.FIREBASE_SERVICE_ACCOUNT)
        firebase_admin.initialize_app(cred)
    return firestore.client()

db = init_firebase()
```

STEP 4: Create FastAPI Main App

4.1 main.py — Entry Point

```
# backend/app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.database import db
from app.routers import jobs, auth, ai, interview
from app.scheduler.job_scheduler import start_scheduler
import uvicorn

app = FastAPI(
    title='HirQube API',
    description='AI-Powered Global Job Platform',
    version='2.0.0'
)

app.add_middleware(CORSMiddleware,
    allow_origins=['*'],
    allow_methods=['*'],
    allow_headers=['*']
)

app.include_router(jobs.router, prefix='/api/jobs', tags=['Jobs'])
app.include_router(auth.router, prefix='/api/auth', tags=['Auth'])
app.include_router(ai.router, prefix='/api/ai', tags=['AI'])

@app.on_event('startup')
async def startup():
    start_scheduler()
    print('HirQube API running. Docs: http://localhost:8000/docs')

@app.get('/api/health')
async def health():
    return {'status': 'live', 'version': '2.0', 'db': 'Firebase'}

if __name__ == '__main__':
    uvicorn.run('main:app', host='0.0.0.0', port=8000, reload=True)
```

Run and Test

```
cd backend
source venv/bin/activate
uvicorn app.main:app --reload --port 8000

# Test in browser:
# http://localhost:8000/api/health
# http://localhost:8000/docs    ← Swagger UI auto-generated!
```

END OF DAY 1 CHECKLIST: FastAPI running ✅ Firebase connected ✅ /docs Swagger UI working ✅ .env configured ✅

💡 **TIP:** Your existing Node.js backend in HirQube-main still works. Keep it running on port 5000 while you build the Python backend on port 8000. You can migrate gradually.

DAY 2 — Multiple Job APIs (500+ Jobs/Day)

Goal: Integrate 8+ real job APIs in Python. Port your existing Node.js services (Adzuna, RemoteOK, GitHub, Dev.to) and add 4 new powerful sources (Jooble, Reed, LinkedIn via RapidAPI, Internshala)

Task	Output
Port Adzuna + RemoteOK to Python	2 sources live
Add Jooble + Reed APIs	4 sources live
Add LinkedIn + Indeed (RapidAPI)	6 sources live
Add Internshala + GitHub Jobs	8 sources live
Deduplication + unified scheduler	500+ jobs/day

API KEYS TO GET (Do This First)

> Sign up for these APIs BEFORE starting. Most are free tier. Get keys now while coding — some take minutes to activate.

Source	Where to Get Key	Free Tier
Adzuna	developer.adzuna.com	**50 req/min FREE**
Jooble	jooble.org/api (email them)	**FREE — email request**
Reed.co.uk	reed.co.uk/developers/jobseeker	**FREE registration**
RemoteOK	remoteok.com/api (no auth needed)	**Fully FREE, no key**
LinkedIn (RapidAPI)	rapidapi.com → search 'LinkedIn Jobs'	**500 req/mo free**
Indeed (RapidAPI)	rapidapi.com → search 'Indeed Jobs'	**500 req/mo free**
Internshala	Scrape via Scrapy (Day 3 covers this)	**FREE scraping**

STEP 1: Unified Job Schema (Pydantic Model)

> All 8 job sources return different field names. This Pydantic model normalizes everything into one standard format.

```
# backend/app/models/job.py
from pydantic import BaseModel, HttpUrl
from typing import Optional, List
from datetime import datetime

class JobSchema(BaseModel):
    # Core
    title: str
```

```
    company: str
    location: str
    description: str = ''
    requirements: str = ''
    apply_url: str
    source: str
    source_id: Optional[str] = None
    fingerprint: Optional[str] = None

    # Dates
    posted_date: Optional[datetime] = None
    last_date: Optional[datetime] = None   # Application deadline

    # Salary
    salary_min: Optional[float] = None
    salary_max: Optional[float] = None
    salary_currency: str = 'USD'
    salary_display: Optional[str] = None

    # Job Type
    job_type: str = 'full-time'      # full-time/part-time/contract/internship
    is_remote: bool = False
    experience_level: str = 'mid'    # entry/mid/senior/lead

    # Company Intelligence
    company_logo: Optional[str] = None
    company_domain: Optional[str] = None
    company_linkedin: Optional[str] = None
    company_size: Optional[str] = None
    company_funding: Optional[str] = None
    company_founded: Optional[int] = None
    glassdoor_rating: Optional[float] = None

    # Recruiter
    recruiter_name: Optional[str] = None
    recruiter_linkedin: Optional[str] = None
    hr_team: List[dict] = []

    # Skills
    required_skills: List[str] = []
    tech_stack: List[str] = []

    created_at: datetime = datetime.utcnow()
```

STEP 2: Adzuna Service (Python Port)

```
# backend/app/services/adzuna_service.py
import httpx, hashlib
from app.config import settings
from app.models.job import JobSchema
from datetime import datetime
from typing import List

async def fetch_adzuna_jobs(keyword='developer', country='us', count=50) ->
List[JobSchema]:
    url = f'https://api.adzuna.com/v1/api/jobs/{country}/search/1'
    params = {
        'app_id': settings.ADZUNA_APP_ID,
        'app_key': settings.ADZUNA_APP_KEY,
```

```
        'results_per_page': count,
        'what': keyword,
        'content-type': 'application/json'
    }
    async with httpx.AsyncClient(timeout=15) as client:
        r = await client.get(url, params=params)
        data = r.json()

    jobs = []
    for j in data.get('results', []):
        fp_str =
f"{j.get('title','')}|{j.get('company',{}).get('display_name','')}|{j.get('location',{}).
get('display_name','')}"
        fingerprint = hashlib.sha256(fp_str.lower().encode()).hexdigest()

        sal_min = j.get('salary_min')
        sal_max = j.get('salary_max')

        jobs.append(JobSchema(
            title=j.get('title',''),
            company=j.get('company',{}).get('display_name','Unknown'),
            location=j.get('location',{}).get('display_name',''),
            description=j.get('description',''),
            apply_url=j.get('redirect_url',''),
            source='adzuna',
            source_id=str(j.get('id','')),
            fingerprint=fingerprint,
            posted_date=datetime.fromisoformat(j['created']).replace(tzinfo=None) if
j.get('created') else None,
            salary_min=float(sal_min) if sal_min else None,
            salary_max=float(sal_max) if sal_max else None,
            salary_display=f'${sal_min:,.0f} - ${sal_max:,.0f}' if sal_min and sal_max
else None,
        ))
    return jobs
```

STEP 3: Jooble Service (New Source)

```
# backend/app/services/jooble_service.py
import httpx, hashlib
from app.config import settings
from app.models.job import JobSchema
from datetime import datetime
from typing import List

JOOBLE_URL = 'https://jooble.org/api/{key}'

async def fetch_jooble_jobs(keyword='developer', location='United States', count=50) ->
List[JobSchema]:
    url = JOOBLE_URL.format(key=settings.JOOBLE_API_KEY)
    payload = {'keywords': keyword, 'location': location, 'page': '1', 'ResultOnPage':
count}
    async with httpx.AsyncClient(timeout=15) as client:
        r = await client.post(url, json=payload)
        data = r.json()

    jobs = []
    for j in data.get('jobs', []):
        fp_str = f"{j.get('title','')}|{j.get('company','')}|{j.get('location','')}"
```

```
        fingerprint = hashlib.sha256(fp_str.lower().encode()).hexdigest()
        jobs.append(JobSchema(
            title=j.get('title',''),
            company=j.get('company',''),
            location=j.get('location',''),
            description=j.get('snippet',''),
            apply_url=j.get('link',''),
            source='jooble',
            source_id=str(j.get('id','')),
            fingerprint=fingerprint,
            salary_display=j.get('salary'),
            posted_date=datetime.fromisoformat(j['updated']) if j.get('updated') else
None,
        ))
    return jobs
```

STEP 4: RapidAPI (LinkedIn + Indeed)

```
# backend/app/services/rapidapi_service.py
import httpx, hashlib
from app.config import settings
from app.models.job import JobSchema
from typing import List

HEADERS = {
    'X-RapidAPI-Key': settings.RAPIDAPI_KEY,
    'X-RapidAPI-Host': 'linkedin-job-search-api.p.rapidapi.com'
}

async def fetch_linkedin_rapidapi(keyword='developer', location='United States') ->
List[JobSchema]:
    url = 'https://linkedin-job-search-api.p.rapidapi.com/active-jd'
    params = {'title': keyword, 'location': location, 'count': '50'}
    async with httpx.AsyncClient(timeout=20) as client:
        r = await client.get(url, headers=HEADERS, params=params)
        data = r.json()

    jobs = []
    for j in data.get('data', []):
        fp_str = f"{j.get('title','')}|{j.get('company','')}|{j.get('location','')}"
        fingerprint = hashlib.sha256(fp_str.lower().encode()).hexdigest()
        jobs.append(JobSchema(
            title=j.get('title',''),
            company=j.get('company',''),
            location=j.get('location',''),
            description=j.get('description',''),
            apply_url=j.get('job_url',''),
            source='linkedin',
            source_id=j.get('id'),
            fingerprint=fingerprint,
            recruiter_name=j.get('hiring_manager_name'),
            recruiter_linkedin=j.get('hiring_manager_linkedin'),
            company_linkedin=j.get('company_linkedin_url'),
        ))
    return jobs
```

STEP 5: Aggregator Service (All Sources)

```
# backend/app/services/job_aggregator.py
import asyncio, logging
from app.services.adzuna_service import fetch_adzuna_jobs
from app.services.jooble_service import fetch_jooble_jobs
from app.services.remoteok_service import fetch_remoteok_jobs
from app.services.rapidapi_service import fetch_linkedin_rapidapi
from app.database import db
from app.models.job import JobSchema

KEYWORDS = ['software engineer','data scientist','product manager',
            'react developer','python developer','devops','UI/UX']
LOCATIONS = ['us','gb','in','ca','au']

async def sync_all_jobs():
    total_new = 0
    total_dup = 0

    for keyword in KEYWORDS:
        tasks = [
            fetch_adzuna_jobs(keyword, 'us'),
            fetch_adzuna_jobs(keyword, 'gb'),
            fetch_adzuna_jobs(keyword, 'in'),
            fetch_jooble_jobs(keyword),
            fetch_remoteok_jobs(keyword),
            fetch_linkedin_rapidapi(keyword),
        ]
        results = await asyncio.gather(*tasks, return_exceptions=True)

        all_jobs = []
        for r in results:
            if isinstance(r, list): all_jobs.extend(r)

        for job in all_jobs:
            inserted = await save_job_if_new(job)
            if inserted: total_new += 1
            else: total_dup += 1

    logging.info(f'Sync complete: {total_new} new, {total_dup} duplicates')
    return {'new': total_new, 'duplicates': total_dup}

async def save_job_if_new(job: JobSchema) -> bool:
    existing = db.collection('jobs').where('fingerprint', '==',
job.fingerprint).limit(1).get()
    if existing: return False
    db.collection('jobs').add(job.dict())
    return True
```

END OF DAY 2: 8 job sources configured ✅ 500+ jobs/day target reachable ✅ Deduplication working ✅ Unified JobSchema ✅

💡 **TIP:** You can test each service individually by calling the async function from a simple Python script before wiring it to the scheduler.

DAY 3 — Automated Scraping + Company Intelligence

Goal: Set up Scrapy spider for Internshala, automate the daily sync with APScheduler, enrich job data with company logos/size/funding/Glassdoor ratings, and extract recruiter & HR LinkedIn profiles

Task	Output
Scrapy setup + Internshala spider	Internship data
APScheduler daily automation	Auto-sync at 2 AM
Company logo + size enrichment	Company profiles
Glassdoor ratings integration	Ratings on jobs
HR/Recruiter profile extraction	LinkedIn profiles

STEP 1: Scrapy Setup

```
# Install Scrapy
pip install scrapy scrapy-splash selenium fake-useragent

# Create Scrapy project inside /scraper
cd hirqube/scraper
scrapy startproject hirqube_scrapers
cd hirqube_scrapers
```

Internshala Spider

```
# scraper/hirqube_scrapers/spiders/internshala.py
import scrapy
from datetime import datetime

class InternshalaSpider(scrapy.Spider):
    name = 'internshala'
    allowed_domains = ['internshala.com']
    start_urls = ['https://internshala.com/internships/']

    custom_settings = {
        'DOWNLOAD_DELAY': 2,          # Be polite: 2 sec between requests
        'CONCURRENT_REQUESTS': 2,
        'ROBOTSTXT_OBEY': True,
        'USER_AGENT': 'HirQube Job Aggregator (contact@hirqube.com)'
    }

    def parse(self, response):
        for card in response.css('.internship_meta'):
            yield {
                'title': card.css('.profile a::text').get('').strip(),
                'company': card.css('.company_name a::text').get('').strip(),
                'location': card.css('.location_link::text').get('Remote').strip(),
                'stipend': card.css('.stipend::text').get('').strip(),
```

```
                'duration': card.css('.internship-other-details
span::text').get('').strip(),
                'apply_url': 'https://internshala.com' + card.css('.profile
a::attr(href)').get(''),
                'source': 'internshala',
                'job_type': 'internship',
                'scraped_at': datetime.utcnow().isoformat()
            }
        # Follow pagination
        next_page = response.css('a.next::attr(href)').get()
        if next_page:
            yield response.follow(next_page, self.parse)

# Run the spider:
# scrapy crawl internshala -o internships.json
```

STEP 2: Daily Automation with APScheduler

```
# backend/app/scheduler/job_scheduler.py
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from apscheduler.triggers.cron import CronTrigger
from app.services.job_aggregator import sync_all_jobs
import logging

scheduler = AsyncIOScheduler()

def start_scheduler():
    # Daily at 2:00 AM UTC
    scheduler.add_job(
        sync_all_jobs,
        CronTrigger(hour=2, minute=0),
        id='daily_job_sync',
        name='Sync all job sources',
        replace_existing=True,
        misfire_grace_time=3600   # Allow 1hr late start
    )

    # Hourly sync for fresh jobs (LinkedIn/RemoteOK only)
    scheduler.add_job(
        sync_fresh_jobs,
        CronTrigger(minute=0),   # Every hour at :00
        id='hourly_fresh_sync',
        name='Hourly fresh job check'
    )

    scheduler.start()
    logging.info('Scheduler started: daily at 2AM + hourly fresh sync')
```

STEP 3: Company Intelligence Enrichment

When a new job is saved, automatically enrich it with company details: logo, size, funding, Glassdoor rating, and LinkedIn HR profiles.

```
# backend/app/services/company_intelligence.py
import httpx
from typing import Optional, Dict
```

```
CLEARBIT_URL = 'https://company.clearbit.com/v2/companies/find'
LOGO_DEV_URL = 'https://img.logo.dev/{domain}?token=pk_free'

async def enrich_company(domain: str) -> Dict:
    result = {}

    # 1. Get company logo
    result['company_logo'] = f'https://logo.clearbit.com/{domain}'

    # 2. Get company details from Clearbit (free tier: 50 lookups/mo)
    async with httpx.AsyncClient(timeout=10) as client:
        try:
            r = await client.get(CLEARBIT_URL,
                params={'domain': domain},
                headers={'Authorization': f'Bearer {settings.CLEARBIT_KEY}'}
            )
            if r.status_code == 200:
                data = r.json()
                result['company_size'] = data.get('metrics', {}).get('employeesRange')
                result['company_funding'] = data.get('metrics', {}).get('raised')
                result['company_founded'] = data.get('foundedYear')
                result['company_linkedin'] = data.get('linkedin', {}).get('handle')
                result['tech_stack'] = data.get('tech', [])[:10]
        except Exception as e:
            pass   # Clearbit failed, continue with what we have

    return result

def extract_domain(url: str) -> Optional[str]:
    from urllib.parse import urlparse
    try:
        return urlparse(url).netloc.replace('www.', '')
    except:
        return None
```

STEP 4: Application Deadline Extraction

Automatically extract 'last date to apply' from job descriptions using regex and NLP patterns.

```
# backend/app/utils/date_extractor.py
import re
from datetime import datetime
from typing import Optional

DATE_PATTERNS = [
    r'apply by[:\s]+([\w\s,]+\d{4})',
    r'deadline[:\s]+([\w\s,]+\d{4})',
    r'last date[:\s]+([\w\s,]+\d{4})',
    r'closing date[:\s]+([\w\s,]+\d{4})',
    r'applications close[:\s]+([\w\s,]+\d{4})',
]

def extract_last_date(description: str) -> Optional[datetime]:
    text = description.lower()
    for pattern in DATE_PATTERNS:
        match = re.search(pattern, text, re.IGNORECASE)
        if match:
            try:
                raw_date = match.group(1).strip()
```

```
            for fmt in ['%B %d, %Y', '%d %B %Y', '%Y-%m-%d', '%d/%m/%Y']:
                try: return datetime.strptime(raw_date, fmt)
                except: continue
        except: pass
    return None
```

END OF DAY 3: Scrapy spider running ✅ Auto-sync at 2AM daily ✅ Company logos + size + funding ✅ Application deadlines extracted ✅ HR LinkedIn profiles captured ✅

💡 **TIP:** For Glassdoor ratings: use RapidAPI's Glassdoor endpoint (search 'Glassdoor' on rapidapi.com). Store ratings in company_ratings Firestore collection and cache for 7 days.

DAYS 4–6 — Authentication System + Firebase Database Schema

Goal: Build complete auth (Email OTP, Google OAuth, Phone OTP), design the full Firebase schema with 100+ collections, and implement JWT token system

DAY 4 — Authentication System

Auth Flow Overview

> HirQube supports 3 auth methods: (1) Email + Password + Verification Link, (2) Google OAuth2 Single Sign-On, (3) Phone Number + Firebase OTP SMS

Install Auth Dependencies

```
pip install python-jose[cryptography] passlib[bcrypt] google-auth
```

Auth Router — Email + Password

```
# backend/app/routers/auth.py
from fastapi import APIRouter, HTTPException, Depends
from fastapi.security import HTTPBearer
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import jwt
from datetime import datetime, timedelta
from app.database import db
from app.config import settings

router = APIRouter()
pwd_ctx = CryptContext(schemes=['bcrypt'])

class RegisterRequest(BaseModel):
    email: EmailStr
    password: str
    full_name: str
    user_type: str   # 'jobseeker' or 'recruiter'

@router.post('/register')
async def register(req: RegisterRequest):
    # Check if email exists
    existing = db.collection('users').where('email','==',req.email).get()
    if existing:
        raise HTTPException(400, 'Email already registered')

    user_id = db.collection('users').document().id
    hashed_pw = pwd_ctx.hash(req.password)

    user_data = {
        'id': user_id,
        'email': req.email,
        'password_hash': hashed_pw,
        'full_name': req.full_name,
        'user_type': req.user_type,
        'is_verified': False,
        'created_at': datetime.utcnow().isoformat()
```

```
    }
    db.collection('users').document(user_id).set(user_data)
    # TODO: Send verification email (Day 5)
    return {'message': 'Registered. Check email to verify.', 'user_id': user_id}

@router.post('/login')
async def login(email: str, password: str):
    users = db.collection('users').where('email','==',email).get()
    if not users:
        raise HTTPException(401, 'Invalid credentials')
    user = users[0].to_dict()
    if not pwd_ctx.verify(password, user['password_hash']):
        raise HTTPException(401, 'Invalid credentials')

    token = jwt.encode({
        'sub': user['id'],
        'type': user['user_type'],
        'exp': datetime.utcnow() + timedelta(days=1)
    }, settings.SECRET_KEY, algorithm='HS256')
    return {'access_token': token, 'token_type': 'bearer'}
```

DAY 5 — Firebase Schema: 100+ Collections

Complete Firestore Schema

> Firebase Firestore is a NoSQL document database. Each collection holds documents. Unlike SQL, Firebase is schema-less but you define it in your code via Pydantic models.

Collection	Purpose	Key Fields
users	All user accounts	id, email, user_type, is_verified
jobseekers	Jobseeker profiles	user_id, skills, experience, education
recruiters	Recruiter profiles	user_id, company_id, is_verified
companies	Company profiles	name, domain, logo, size, linkedin
jobs	All job listings	title, company, source, fingerprint
internships	Internship listings	title, stipend, duration, source
applications	Job applications	user_id, job_id, status, ats_score
resumes	Uploaded resumes	user_id, file_url, ats_score, parsed_data
ats_scores	ATS analysis results	resume_id, job_id, score, gaps
interviews	AI interview sessions	user_id, job_id, scores, recording
certifications	User certificates	user_id, name, issuer, verify_url
coding_profiles	LeetCode/HackerRank	user_id, platform, rating, solved

skills	Skills taxonomy	name, category, related_jobs
skill_gaps	Gap analysis results	user_id, missing_skills, roadmap
notifications	All notifications	user_id, type, message, is_read
job_recommendations	AI recommendations	user_id, job_id, score, reason
hackathons	Hackathon listings	name, platform, deadline, prizes
scraping_logs	Scraper audit trail	source, run_date, jobs_found
verification_requests	Recruiter verification	recruiter_id, company_domain, status
email_templates	Email content	type, subject, html_body
api_keys	API key management	key_hash, owner_id, rate_limit
audit_logs	Security audit	user_id, action, ip, timestamp
reports	Fraud reports	reporter_id, job_id, reason, status

Firestore Security Rules

```
// firestore.rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can only read their own data
    match /users/{userId} {
      allow read, write: if request.auth.uid == userId;
    }
    // Jobs are public read, server-only write
    match /jobs/{jobId} {
      allow read: if true;
      allow write: if request.auth.token.admin == true;
    }
    // Applications: only the owner
    match /applications/{appId} {
      allow read, write: if request.auth.uid == resource.data.user_id;
    }
  }
}
```

DAY 6 — Email OTP + Phone OTP + Google OAuth

Email Verification (SendGrid)

```
pip install sendgrid

# backend/app/services/email_service.py
import sendgrid
from sendgrid.helpers.mail import Mail
import secrets

async def send_verification_email(email: str, user_id: str):
    token = secrets.token_urlsafe(32)
```

```
    # Store token in Firestore
    db.collection('email_verifications').add({
        'user_id': user_id, 'token': token,
        'expires_at': (datetime.utcnow() + timedelta(hours=24)).isoformat()
    })
    verify_url = f'https://hirqube.com/verify?token={token}'

    sg = sendgrid.SendGridAPIClient(api_key=settings.SENDGRID_KEY)
    mail = Mail(
        from_email='noreply@hirqube.com',
        to_emails=email,
        subject='Verify your HirQube account',
        html_content=f'<a href="{verify_url}">Click to verify</a>'
    )
    sg.client.mail.send.post(request_body=mail.get())
```

Phone OTP (Firebase Auth)

```
# Use Firebase Authentication on the frontend
# React: firebase.auth().signInWithPhoneNumber(phoneNumber, recaptchaVerifier)

# Backend: verify the Firebase ID token
from firebase_admin import auth as firebase_auth

@router.post('/verify-phone-token')
async def verify_phone_token(id_token: str):
    decoded = firebase_auth.verify_id_token(id_token)
    phone = decoded.get('phone_number')
    uid = decoded.get('uid')
    # Link phone to your user record
    return {'verified': True, 'phone': phone, 'uid': uid}
```

END OF DAYS 4-6: Full auth system ✅ Firebase 24+ collections designed ✅ JWT tokens ✅ Email verification ✅ Phone OTP ✅

DAYS 7–9 — Company Intelligence & Rich Job Data

Goal: Enrich every job with: company logo (proper, not favicon), company size/funding/founded year, Glassdoor rating, tech stack used, and recruiter + HR team LinkedIn profiles

DAY 7 — Company Profiles

Data Sources for Company Intelligence

Data	Source / API	Free / Paid
Company Logo	logo.clearbit.com/{domain}	**FREE**
Company Size	Clearbit Company API	**50 lookups/mo free**
Funding Info	Crunchbase API	**$29/mo basic plan**
Glassdoor Rating	RapidAPI Glassdoor endpoint	**500 req/mo free**
Tech Stack	Clearbit + job description NLP	**FREE via NLP**
HR LinkedIn Profiles	RapidAPI LinkedIn People Search	**Shared RapidAPI key**

Company Enrichment Pipeline

```
# backend/app/services/company_enricher.py
import httpx
from app.database import db

async def enrich_company_full(company_name: str, domain: str) -> dict:
    company_ref = db.collection('companies').where('domain','==',domain).get()
    if company_ref:
        return company_ref[0].to_dict()   # Use cached data

    data = {
        'name': company_name,
        'domain': domain,
        'logo': f'https://logo.clearbit.com/{domain}',
    }

    # Clearbit enrichment
    async with httpx.AsyncClient(timeout=10) as client:
        r = await
client.get(f'https://company.clearbit.com/v2/companies/find?domain={domain}',
            headers={'Authorization': f'Bearer {settings.CLEARBIT_KEY}'})
        if r.status_code == 200:
            cb = r.json()
            data.update({
                'size': cb.get('metrics',{}).get('employeesRange'),
                'founded': cb.get('foundedYear'),
                'linkedin': cb.get('linkedin',{}).get('handle'),
                'tech_stack': [t['tag'] for t in cb.get('tech',[])[:15]],
                'description': cb.get('description',''),
                'type': cb.get('type'),
            })

    # Glassdoor rating
    data['glassdoor_rating'] = await get_glassdoor_rating(company_name)
```

```
    # HR team (LinkedIn)
    data['hr_team'] = await get_hr_team(company_name, domain)

    # Cache in Firestore
    db.collection('companies').add(data)
    return data

async def get_hr_team(company: str, domain: str) -> list:
    # Search LinkedIn for HR/Recruiter profiles at this company
    url = 'https://linkedin-people-search.p.rapidapi.com/search'
    headers = {'X-RapidAPI-Key': settings.RAPIDAPI_KEY,
               'X-RapidAPI-Host': 'linkedin-people-search.p.rapidapi.com'}
    params = {'query': f'HR recruiter talent acquisition {company}', 'count': 5}
    async with httpx.AsyncClient(timeout=10) as client:
        r = await client.get(url, headers=headers, params=params)
        if r.status_code == 200:
            people = r.json().get('data', [])
            return [{'name': p.get('name'), 'title': p.get('title'),
                     'linkedin_url': p.get('profile_url')} for p in people[:5]]
    return []
```

END OF DAYS 7-9: Company logos (proper CDN) ✅ Company size/funding/founded ✅ Glassdoor ratings ✅ Tech stack ✅ HR LinkedIn profiles ✅

DAYS 10–13 — AI Core: ATS Scoring, Resume Parser & Skill Gap

Goal: Build the ATS scoring engine (0–100), resume PDF/DOCX parser using SpaCy NLP, skill extraction, gap analysis, job-resume match scoring with Sentence Transformers, and AI improvement suggestions via Claude API

DAY 10 — Resume Parser (PDF + DOCX → Structured Data)

Resume Parsing Strategy

> Your resume parser must extract: Contact Info, Education (degree, institution, year), Work Experience (company, role, dates, description), Skills (technical + soft), Projects, and Certifications.

```
# backend/app/ai/resume_parser.py
import spacy
import pdfplumber
import docx
import re
from typing import Dict, List

nlp = spacy.load('en_core_web_lg')

TECH_SKILLS = {'python','javascript','react','nodejs','typescript','java',
               'sql','postgresql','mongodb','docker','kubernetes','aws',
               'git','linux','fastapi','django','machine learning','nlp',
               'tensorflow','pytorch','scikit-learn','pandas','numpy',
               'html','css','tailwind','flutter','dart','firebase'}

def extract_text_from_pdf(file_bytes: bytes) -> str:
    import io
    with pdfplumber.open(io.BytesIO(file_bytes)) as pdf:
        return '\n'.join(p.extract_text() or '' for p in pdf.pages)

def extract_text_from_docx(file_bytes: bytes) -> str:
    import io
    doc = docx.Document(io.BytesIO(file_bytes))
    return '\n'.join(p.text for p in doc.paragraphs)

def parse_resume(file_bytes: bytes, file_type: str) -> Dict:
    text = extract_text_from_pdf(file_bytes) if file_type=='pdf' else
extract_text_from_docx(file_bytes)
    doc = nlp(text)

    return {
        'raw_text': text,
        'email': extract_email(text),
        'phone': extract_phone(text),
        'name': extract_name(doc),
        'skills': extract_skills(text),
        'education': extract_education(text),
        'experience': extract_experience(text),
        'total_experience_years': calc_experience_years(text),
    }

def extract_email(text: str) -> str:
```

```
    match = re.search(r'[\w.+-]+@[\w-]+\.[\w.]+', text)
    return match.group(0) if match else ''

def extract_phone(text: str) -> str:
    match = re.search(r'[\+]?[(]?[0-9]{3}[)]?[-\s\.]?[0-9]{3}[-\s\.]?[0-9]{4,6}', text)
    return match.group(0) if match else ''

def extract_name(doc) -> str:
    for ent in doc.ents:
        if ent.label_ == 'PERSON': return ent.text
    return ''

def extract_skills(text: str) -> List[str]:
    text_lower = text.lower()
    return [s for s in TECH_SKILLS if s in text_lower]
```

DAY 11 — ATS Scoring Engine (0–100)

ATS Scoring Formula

ATS Component	Weight	Max Points
Keyword Match (TF-IDF + embeddings)	**35%**	35
Skills Overlap	**25%**	25
Experience Match	**20%**	20
Education Match	**10%**	10
Resume Format Score	**10%**	10

```
# backend/app/ai/ats_scorer.py
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

def calculate_ats_score(resume_text: str, job_description: str,
                        resume_skills: list, job_skills: list,
                        resume_exp_years: float, job_exp_required: float) -> dict:

    # 1. Semantic similarity (35 points)
    emb_resume = model.encode([resume_text[:1000]])
    emb_job    = model.encode([job_description[:1000]])
    semantic_sim = cosine_similarity(emb_resume, emb_job)[0][0]
    semantic_score = round(semantic_sim * 35, 1)

    # 2. Skills overlap (25 points)
    resume_set = set(s.lower() for s in resume_skills)
    job_set    = set(s.lower() for s in job_skills)
    if job_set:
        overlap = len(resume_set & job_set) / len(job_set)
        skills_score = round(overlap * 25, 1)
    else:
        skills_score = 12.5
```

```
    # 3. Experience match (20 points)
    if job_exp_required > 0:
        exp_ratio = min(resume_exp_years / job_exp_required, 1.2)
        exp_score = round(min(exp_ratio, 1.0) * 20, 1)
    else:
        exp_score = 15.0

    # 4. Education (10 points) — simplified
    edu_score = 7.0   # Base, improve later with degree matching

    # 5. Format score (10 points)
    format_score = score_resume_format(resume_text)

    total = semantic_score + skills_score + exp_score + edu_score + format_score
    total = round(min(total, 100), 1)

    missing_skills = list(job_set - resume_set)

    return {
        'score': total,
        'breakdown': {
            'semantic': semantic_score,
            'skills': skills_score,
            'experience': exp_score,
            'education': edu_score,
            'format': format_score
        },
        'missing_skills': missing_skills,
        'grade': 'A' if total>=85 else 'B' if total>=70 else 'C' if total>=55 else 'D'
    }
```

DAY 12–13 — AI Resume Improvement (Claude API)

```
# backend/app/ai/resume_improver.py
import anthropic
from app.config import settings

client = anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

async def get_resume_suggestions(resume_text: str, job_description: str, ats_score:
float) -> dict:
    prompt = f'''You are a professional resume coach and ATS expert.

Resume (current ATS score: {ats_score}/100):
{resume_text[:2000]}

Job Description:
{job_description[:1500]}

Provide specific, actionable improvement suggestions as JSON:
{{
  "summary_suggestions": ["..."],
  "skills_to_add": ["..."],
  "experience_rewrites": ["Before: ... After: ..."],
  "keywords_to_include": ["..."],
  "format_improvements": ["..."],
  "predicted_new_score": 85
```

```
}}'''

    message = client.messages.create(
        model='claude-sonnet-4-20250514',
        max_tokens=1500,
        messages=[{'role': 'user', 'content': prompt}]
    )

    import json
    try:
        return json.loads(message.content[0].text)
    except:
        return {'error': 'Could not parse suggestions', 'raw': message.content[0].text}
```

END OF DAYS 10-13: Resume parser (PDF + DOCX) ✅ ATS scoring 0-100 ✅ Skill extraction ✅ Gap analysis ✅ Claude AI improvement suggestions ✅

DAYS 14–17 — React Frontend: Jobseeker & Recruiter Dashboards

Goal: Build complete React frontend with Tailwind CSS — Jobseeker dashboard (job search, ATS checker, profile), Recruiter portal (post jobs, candidate search, AI ranking), and Admin panel

DAY 14 — React Project Setup

Create React App with Tailwind

```
cd hirqube/frontend
npx create-react-app . --template typescript
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
npm install axios react-router-dom react-query firebase
npm install @headlessui/react @heroicons/react framer-motion
npm install recharts react-dropzone react-toastify
```

Project Structure

```
frontend/src/
├── components/
│   ├── JobCard.tsx           ← Enhanced from existing
│   ├── JobDetail.tsx         ← Enhanced from existing
│   ├── ATSScoreCircle.tsx    ← New: animated score display
│   ├── ResumeUploader.tsx    ← New: drag & drop upload
│   ├── SkillGapChart.tsx     ← New: gap visualization
│   └── CompanyCard.tsx       ← New: company details
├── pages/
│   ├── JobsPage.tsx          ← Enhanced from existing
│   ├── JobseekerDashboard.tsx
│   ├── RecruiterDashboard.tsx
│   ├── ProfilePage.tsx
│   ├── ATSChecker.tsx
│   ├── InterviewPractice.tsx
│   └── AdminPanel.tsx
├── services/
│   └── api.ts                ← Enhanced from existing
└── App.tsx
```

DAY 15 — ATS Score Display Component

```
// frontend/src/components/ATSScoreCircle.tsx
import { motion } from 'framer-motion';

interface Props { score: number; label?: string; }

export default function ATSScoreCircle({ score, label }: Props) {
  const color = score >= 85 ? '#1E8449' : score >= 70 ? '#2E86C1' : score >= 55 ?
'#D35400' : '#E74C3C';
  const radius = 54;
  const circumference = 2 * Math.PI * radius;
  const offset = circumference - (score / 100) * circumference;

  return (
```

```
    <div className='flex flex-col items-center'>
      <svg width='140' height='140' viewBox='0 0 140 140'>
        <circle cx='70' cy='70' r='54' fill='none' stroke='#E8E8E8' strokeWidth='10'/>
        <motion.circle cx='70' cy='70' r='54' fill='none'
          stroke={color} strokeWidth='10'
          strokeDasharray={circumference}
          strokeDashoffset={offset}
          strokeLinecap='round'
          transform='rotate(-90 70 70)'
          initial={{ strokeDashoffset: circumference }}
          animate={{ strokeDashoffset: offset }}
          transition={{ duration: 1.5, ease: 'easeOut' }}
        />
        <text x='70' y='68' textAnchor='middle' fontSize='28' fontWeight='bold'
fill={color}>{score}</text>
        <text x='70' y='88' textAnchor='middle' fontSize='11' fill='#666'>/100</text>
      </svg>
      {label && <span className='mt-2 text-sm font-semibold
text-gray-600'>{label}</span>}
    </div>
  );
}
```

DAY 16 — Jobseeker Dashboard

```
// frontend/src/pages/JobseekerDashboard.tsx  (simplified)
import { useState, useEffect } from 'react';
import ATSScoreCircle from '../components/ATSScoreCircle';

export default function JobseekerDashboard() {
  const [applications, setApplications] = useState([]);
  const [resumeScore, setResumeScore] = useState(0);

  return (
    <div className='min-h-screen bg-gray-50'>
      {/* Header */}
      <nav className='bg-white shadow-sm px-6 py-4 flex items-center justify-between'>
        <span className='text-2xl font-bold text-blue-900'>HirQube</span>
        <div className='flex gap-4'>
          <button className='text-blue-600 hover:text-blue-800'>Jobs</button>
          <button className='text-blue-600 hover:text-blue-800'>Applications</button>
          <button className='text-blue-600 hover:text-blue-800'>ATS Check</button>
          <button className='text-blue-600 hover:text-blue-800'>Interview Prep</button>
        </div>
      </nav>

      <div className='max-w-7xl mx-auto px-6 py-8 grid grid-cols-4 gap-6'>

        {/* Stats Cards */}
        <div className='col-span-4 grid grid-cols-4 gap-4'>
          {[
            { label: 'Applications Sent', value: applications.length, color: 'blue' },
            { label: 'Interviews Scheduled', value: 3, color: 'green' },
            { label: 'Resume ATS Score', value: `${resumeScore}/100`, color: 'purple' },
            { label: 'Profile Completeness', value: '85%', color: 'orange' },
          ].map(s => (
            <div key={s.label} className='bg-white rounded-xl shadow-sm p-6'>
              <p className='text-sm text-gray-500'>{s.label}</p>
```

```
              <p className='text-3xl font-bold text-gray-800 mt-1'>{s.value}</p>
            </div>
          ))}
        </div>

        {/* Main: Job Recommendations */}
        <div className='col-span-3'>
          <h2 className='text-xl font-bold mb-4'>Recommended for You</h2>
          {/* JobCard components mapped here */}
        </div>

        {/* Sidebar: ATS Score */}
        <div className='col-span-1'>
          <div className='bg-white rounded-xl shadow-sm p-6 text-center'>
            <h3 className='font-bold mb-4'>Resume Score</h3>
            <ATSScoreCircle score={resumeScore} label='ATS Score' />
          </div>
        </div>
      </div>
    </div>
  );
}
```

DAY 17 — Job Card Enhancement (Rich Data)

Enhance your existing JobCard component to show: company logo (proper CDN), Glassdoor rating stars, tech stack tags, application deadline, Glassdoor salary estimate, and recruiter LinkedIn.

```
// Key additions to existing JobCard.tsx
// Company Logo with fallback:
<img src={job.company_logo || `https://logo.clearbit.com/${job.company_domain}`}
     onError={(e) => e.currentTarget.src = '/placeholder-company.png'}
     className='w-12 h-12 rounded-lg object-contain border'
     alt={job.company} />

// Glassdoor rating:
{job.glassdoor_rating && (
  <div className='flex items-center gap-1 text-sm'>
    <span className='text-yellow-500'>★</span>
    <span>{job.glassdoor_rating}/5 Glassdoor</span>
  </div>
)}

// Application deadline badge:
{job.last_date && (
  <span className='bg-red-100 text-red-700 text-xs px-2 py-1 rounded-full'>
    Apply by {new Date(job.last_date).toLocaleDateString()}
  </span>
)}

// Tech stack tags:
{job.tech_stack?.slice(0,4).map(tech => (
  <span key={tech} className='bg-blue-50 text-blue-700 text-xs px-2 py-1
rounded'>{tech}</span>
))}

// Recruiter LinkedIn link:
{job.recruiter_linkedin && (
```

```
  <a href={job.recruiter_linkedin} target='_blank'
     className='text-blue-600 text-sm flex items-center gap-1'>
    Connect with Recruiter on LinkedIn →
  </a>
)}
```

END OF DAYS 14-17: React setup with TypeScript ✅ ATS score circle ✅ Jobseeker dashboard ✅ Recruiter portal ✅ Rich job cards with logos/ratings ✅

DAYS 18–21 — Neural Interview Engine (AI First Round)

Goal: Build the complete AI interview system — question generation, speech-to-text, NLP response analysis, scoring (Communication 30%, Technical 40%, Confidence 15%, Problem Solving 15%), and automated interview report generation

Interview System Architecture

Flow: Recruiter shortlists candidate → System sends interview invite → Candidate joins AI interview → AI asks questions in sequence → Speech captured → NLP analysis → Score + Report → Recruiter reviews recommendation

Scoring Breakdown

Component	Weight	How Measured
Technical Knowledge	**40%**	NLP keyword accuracy vs expected answer
Communication	**30%**	Sentence structure, vocabulary, clarity
Confidence	**15%**	Hesitation words, speech pace, filler words
Problem Solving	**15%**	Structured thinking, STAR method usage

DAY 18 — Question Generator

```
# backend/app/ai/interview_question_generator.py
import anthropic
from app.config import settings

client = anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

async def generate_interview_questions(job_title: str, job_description: str,
                                       resume_text: str) -> dict:
    prompt = f'''Generate a structured interview for: {job_title}

Job Description: {job_description[:800]}
Candidate Resume Summary: {resume_text[:500]}

Generate exactly these questions as JSON:
{{
  "self_intro": "Tell me about yourself and your journey to {job_title}.",
  "technical": [
    {{"question": "...", "expected_keywords": ["k1","k2"], "difficulty": "medium"}},
    {{"question": "...", "expected_keywords": ["k1","k2"], "difficulty": "hard"}},
    {{"question": "...", "expected_keywords": ["k1","k2"], "difficulty": "medium"}}
  ],
  "aptitude": [
    {{"question": "...", "type": "logical"}},
    {{"question": "...", "type": "analytical"}}
  ],
```

```
  "behavioral": [
    {{"question": "Describe a challenging situation and how you resolved it."}},
    {{"question": "Tell me about a project you are most proud of."}}
  ]
}}'''

    msg = client.messages.create(
        model='claude-sonnet-4-20250514',
        max_tokens=1500,
        messages=[{'role': 'user', 'content': prompt}]
    )
    import json
    return json.loads(msg.content[0].text)
```

DAY 19 — Speech-to-Text Integration

```
# Use Google Speech-to-Text (free tier: 60 min/month)
# Or OpenAI Whisper (open-source, local)
pip install openai-whisper

# backend/app/ai/speech_transcriber.py
import whisper
import tempfile, os

whisper_model = whisper.load_model('base')  # 74MB, runs on CPU

async def transcribe_audio(audio_bytes: bytes, audio_format: str = 'webm') -> dict:
    with tempfile.NamedTemporaryFile(suffix=f'.{audio_format}', delete=False) as f:
        f.write(audio_bytes)
        tmp_path = f.name

    try:
        result = whisper_model.transcribe(tmp_path)
        return {
            'text': result['text'],
            'language': result['language'],
            'confidence': result.get('confidence', 1.0)
        }
    finally:
        os.unlink(tmp_path)
```

DAY 20 — NLP Response Analyzer

```
# backend/app/ai/interview_analyzer.py
import spacy, re
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

nlp = spacy.load('en_core_web_lg')
model = SentenceTransformer('all-MiniLM-L6-v2')

FILLER_WORDS = {'um','uh','like','you know','basically','literally','actually','right'}

def analyze_response(response_text: str, expected_keywords: list) -> dict:
    doc = nlp(response_text.lower())
    word_count = len([t for t in doc if not t.is_punct])
```

```
    sent_count = len(list(doc.sents))

    # Filler word analysis
    filler_count = sum(response_text.lower().count(f) for f in FILLER_WORDS)
    filler_rate = filler_count / max(word_count, 1)

    # Keyword coverage
    keywords_found = [kw for kw in expected_keywords if kw.lower() in
response_text.lower()]
    keyword_score = len(keywords_found) / max(len(expected_keywords), 1)

    # Semantic similarity to expected answer
    if expected_keywords:
        expected_str = ' '.join(expected_keywords)
        emb1 = model.encode([response_text[:500]])
        emb2 = model.encode([expected_str])
        semantic_score = float(cosine_similarity(emb1, emb2)[0][0])
    else:
        semantic_score = 0.5

    # Communication quality (sentence length variance, vocabulary)
    avg_sent_len = word_count / max(sent_count, 1)
    comm_score = min(1.0, (1 - filler_rate) * 0.6 + min(avg_sent_len/15, 1.0) * 0.4)

    # Confidence (based on positive assertive language)
    confidence_indicators = ['i believe','i think','in my experience','i have','i
developed']
    conf_count = sum(1 for c in confidence_indicators if c in response_text.lower())
    conf_score = min(1.0, (1 - filler_rate * 2) * 0.5 + min(conf_count/3, 1.0) * 0.5)

    return {
        'technical_score': round(keyword_score * 40, 1),
        'communication_score': round(comm_score * 30, 1),
        'confidence_score': round(conf_score * 15, 1),
        'problem_solving_score': round(semantic_score * 15, 1),
        'total': round(keyword_score*40 + comm_score*30 + conf_score*15 +
semantic_score*15, 1),
        'keywords_found': keywords_found,
        'word_count': word_count,
        'filler_word_rate': round(filler_rate * 100, 1),
    }
```

DAY 21 — Interview Report Generator

```
# backend/app/ai/interview_report.py
async def generate_interview_report(interview_session: dict) -> dict:
    scores = interview_session['scores']
    total = sum(s['total'] for s in scores) / len(scores)

    prompt = f'''You are an HR expert reviewing an AI interview.
    Overall score: {total:.1f}/100
    Technical: {scores[0]['technical_score']}  Communication:
{scores[0]['communication_score']}
    Confidence: {scores[0]['confidence_score']}  Problem Solving:
{scores[0]['problem_solving_score']}

    Write a concise interview report as JSON:
    {{"recommendation": "Strong Hire|Hire|Maybe|No Hire",
      "strengths": ["..."],
```

```
    "improvements": ["..."],
    "summary": "2-3 sentence summary for recruiter"}}'''

  msg = client.messages.create(
      model='claude-sonnet-4-20250514', max_tokens=600,
      messages=[{'role': 'user', 'content': prompt}]
  )
  import json
  report = json.loads(msg.content[0].text)
  report['total_score'] = round(total, 1)
  return report
```

END OF DAYS 18-21: Question generator ✅ Whisper speech-to-text ✅ NLP response analyzer ✅ 4-component scoring ✅ Interview report with hire recommendation ✅

DAYS 22–25 — Flutter Mobile App + Notifications System

Goal: Build Flutter mobile app (iOS + Android), integrate Firebase push notifications, add daily job digest emails (SendGrid), and SMS alerts via Twilio for interview invites

DAY 22 — Flutter Project Setup

Install Flutter

```
# Install Flutter SDK (https://flutter.dev/docs/get-started/install)
# Add to PATH and verify:
flutter doctor

# Create Flutter project
cd hirqube/mobile
flutter create hirqube_app --org com.hirqube
cd hirqube_app

# Add dependencies to pubspec.yaml:
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  firebase_core: ^2.24.0
  firebase_auth: ^4.15.0
  firebase_messaging: ^14.7.0
  go_router: ^12.0.0
  provider: ^6.1.1
  cached_network_image: ^3.3.0
  flutter_local_notifications: ^16.2.0
  file_picker: ^6.1.0
```

DAY 23 — Mobile Job List Screen

```
// mobile/lib/screens/jobs_screen.dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class JobsScreen extends StatefulWidget {
  @override _JobsScreenState createState() => _JobsScreenState();
}

class _JobsScreenState extends State<JobsScreen> {
  List jobs = [];
  bool loading = true;
  String query = 'developer';

  @override
  void initState() { super.initState(); fetchJobs(); }

  Future fetchJobs() async {
    final res = await http.get(Uri.parse(
      'https://api.hirqube.com/api/jobs?q=$query&limit=20'));
    setState(() {
```

```
      jobs = json.decode(res.body)['data'];
      loading = false;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('HirQube', style: TextStyle(fontWeight: FontWeight.bold)),
        backgroundColor: Color(0xFF1B4F72),
        foregroundColor: Colors.white,
      ),
      body: loading ? Center(child: CircularProgressIndicator())
        : ListView.builder(
          itemCount: jobs.length,
          itemBuilder: (ctx, i) => JobCard(job: jobs[i]),
        ),
    );
  }
}
```

DAY 24 — Firebase Push Notifications

Backend: Send Push Notification

```
# backend/app/services/notification_service.py
from firebase_admin import messaging

async def send_push_notification(fcm_token: str, title: str, body: str, data: dict = {}):
    message = messaging.Message(
        notification=messaging.Notification(title=title, body=body),
        data={k: str(v) for k, v in data.items()},
        token=fcm_token,
        android=messaging.AndroidConfig(priority='high'),
        apns=messaging.APNSConfig(
            payload=messaging.APNSPayload(
                aps=messaging.Aps(sound='default', badge=1)
            )
        )
    )
    response = messaging.send(message)
    return {'message_id': response}

# Use cases:
# await send_push_notification(token, 'New Job Match!',
#   'React Developer at Google — 92% match', {'job_id': '123'})
#
# await send_push_notification(token, 'Interview Scheduled',
#   'AI Interview for Senior Dev role — starts in 10 min', {'interview_id': '456'})
```

DAY 25 — Daily Email Digest (SendGrid)

```
# backend/app/services/email_digest.py
# Runs daily at 8 AM — sends personalized job digest to each jobseeker

async def send_daily_job_digest():
```

```
    jobseekers = db.collection('users').where('user_type','==','jobseeker')
    # Build personalized digest for each user
    for user_doc in jobseekers.stream():
        user = user_doc.to_dict()
        if not user.get('email_notifications', True): continue

        # Get top 5 new jobs matching user skills
        skills = user.get('skills', ['developer'])
        query = ' OR '.join(skills[:3])
        new_jobs = (db.collection('jobs')
            .where('created_at', '>', yesterday_timestamp)
            .limit(5).get())

        if not new_jobs: continue

        html = build_email_html(user['full_name'], [j.to_dict() for j in new_jobs])
        await send_email(user['email'], 'Your Daily Job Digest — HirQube', html)

def build_email_html(name: str, jobs: list) -> str:
    job_html = ''.join(f'''
    <div style='border:1px solid #ddd;padding:16px;margin:8px 0;border-radius:8px;'>
      <img src='{j.get("company_logo","")}' width='40' height='40'
style='border-radius:4px;'>
      <strong>{j["title"]}</strong> at {j["company"]}<br>
      {j["location"]} | {j.get("salary_display","Salary not specified")}<br>
      <a href='https://hirqube.com/jobs/{j["id"]}'>View & Apply →</a>
    </div>''' for j in jobs)
    return f'<h2>Hi {name}, here are today\'s top jobs for you!</h2>{job_html}'
```

END OF DAYS 22-25: Flutter app setup ✅ Mobile job list ✅ Firebase push notifications ✅ Daily email digest ✅ SMS alerts (Twilio) ready ✅

DAYS 26–30 — Performance, Scaling & AWS Deployment

Goal: Optimize search with Algolia/Elasticsearch full-text search, add Redis caching layer, containerize with Docker, deploy on AWS (EC2 + S3 + CloudFront), set up CI/CD, configure monitoring

DAY 26 — Redis Caching Layer

```
# backend/app/cache.py
import redis.asyncio as redis
import json
from app.config import settings

redis_client = redis.from_url(settings.REDIS_URL)

async def cache_get(key: str):
    val = await redis_client.get(key)
    return json.loads(val) if val else None

async def cache_set(key: str, value, ttl: int = 300):
    await redis_client.setex(key, ttl, json.dumps(value, default=str))

# Usage in jobs router:
@router.get('/jobs')
async def get_jobs(q: str = '', location: str = '', page: int = 1):
    cache_key = f'jobs:{q}:{location}:{page}'
    cached = await cache_get(cache_key)
    if cached:
        return cached   # Return from Redis in <1ms

    # Fetch from Firestore
    jobs = await fetch_jobs_from_db(q, location, page)
    await cache_set(cache_key, jobs, ttl=600)   # Cache 10 min
    return jobs
```

DAY 27 — Algolia Search Integration

Algolia provides instant full-text search across 1M+ jobs with faceted filtering (salary, skills, location, job type) in under 50ms.

```
pip install algoliasearch

# backend/app/services/search_service.py
from algoliasearch.search_client import SearchClient
from app.config import settings

client = SearchClient.create(settings.ALGOLIA_APP_ID, settings.ALGOLIA_API_KEY)
index = client.init_index('jobs')

# Configure index (run once)
def setup_algolia_index():
    index.set_settings({
        'searchableAttributes': ['title', 'company', 'location', 'required_skills',
'description'],
```

```
        'attributesForFaceting': ['source', 'job_type', 'location', 'required_skills',
'is_remote'],
        'ranking': ['desc(posted_date)', 'typo', 'geo', 'words', 'filters', 'proximity',
'attribute', 'exact', 'custom'],
        'customRanking': ['desc(glassdoor_rating)'],
    })

async def search_jobs(query: str, filters: dict = {}, page: int = 0) -> dict:
    algolia_filters = []
    if filters.get('job_type'): algolia_filters.append(f'job_type:{filters["job_type"]}')
    if filters.get('is_remote'): algolia_filters.append('is_remote:true')
    if filters.get('location'):
algolia_filters.append(f'location:"{filters["location"]}"')

    results = index.search(query, {
        'filters': ' AND '.join(algolia_filters),
        'page': page,
        'hitsPerPage': 20,
        'facets': ['source', 'job_type', 'location'],
    })
    return results

# Sync a new job to Algolia:
def index_job(job: dict):
    index.save_object({**job, 'objectID': job['id']})
```

DAY 28 — Docker Containerization

Dockerfile (Backend)

```
# backend/Dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN python -m spacy download en_core_web_lg

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose.yml

```
version: '3.9'
services:
  backend:
    build: ./backend
    ports: ['8000:8000']
    env_file: ./backend/.env
    depends_on: [redis]
    restart: unless-stopped

  frontend:
    build: ./frontend
    ports: ['3000:3000']
    restart: unless-stopped
```

```
  redis:
    image: redis:7-alpine
    ports: ['6379:6379']
    volumes: ['redis_data:/data']

  scraper:
    build: ./scraper
    depends_on: [backend]

volumes:
  redis_data:
```

DAY 29 — AWS Deployment

AWS Architecture

Service	AWS Component	Cost (approx)
Backend API	EC2 t3.medium + Auto Scaling	$30–60/mo
Frontend	S3 + CloudFront CDN	$5–15/mo
Database	Firebase (already cloud)	Free–$25/mo
Cache	ElastiCache Redis	$13–30/mo
File Storage	S3 (resumes, logos)	$1–5/mo
Search	Algolia (10K queries free)	Free–$29/mo
CI/CD	GitHub Actions	Free
Monitoring	CloudWatch + Sentry	Free–$10/mo

AWS Deployment Steps

8. Create AWS account at aws.amazon.com
9. Launch EC2 t3.medium instance (Ubuntu 22.04)
10. SSH in: ssh -i hirqube.pem ubuntu@YOUR_EC2_IP
11. Install Docker: curl -fsSL https://get.docker.com | sh
12. Clone your repo: git clone https://github.com/you/hirqube
13. Copy .env file to server (never commit this to Git!)
14. Run: docker compose up -d
15. Configure Nginx reverse proxy (port 80/443 → 8000)
16. Get SSL certificate: sudo certbot --nginx
17. Build frontend: cd frontend && npm run build
18. Upload to S3: aws s3 sync build/ s3://hirqube-frontend
19. Create CloudFront distribution pointing to S3 bucket
20. Update all API URLs in frontend to production domain

DAY 30 — Monitoring & Final Checks

```
# Install Sentry for error tracking
```

```
pip install sentry-sdk[fastapi]

# In main.py:
import sentry_sdk
sentry_sdk.init(dsn=settings.SENTRY_DSN, traces_sample_rate=0.1)
```

Production Readiness Checklist

- API rate limiting (slowapi: pip install slowapi)
- HTTPS enabled on all endpoints
- CORS restricted to your domain only (not '*')
- Firebase security rules tested
- All API keys in environment variables (not code)
- Database backups configured
- Error logging with Sentry
- Health endpoint monitored (UptimeRobot free tier)
- Daily job sync verified working
- Mobile app submitted to Play Store / App Store

> 🚀 LAUNCH DAY 30: Platform deployed on AWS ✅ Search with Algolia ✅ Redis caching ✅ Docker containers ✅ CI/CD pipeline ✅ Monitoring live ✅ HirQube is LIVE!

QUICK REFERENCE — Commands, APIs & Secrets

Essential Commands

Command	What it does
uvicorn app.main:app --reload	Start FastAPI backend (dev)
docker compose up -d	Start all services
scrapy crawl internshala	Run internship scraper
python -m pytest tests/	Run all tests
npm run build (frontend)	Build React for production
flutter build apk --release	Build Android APK
firebase deploy --only firestore:rules	Deploy Firestore rules
aws s3 sync build/ s3://hirqube	Deploy frontend to AWS S3

All API Keys Needed

Service	Sign Up URL	Cost
JOB DATA SOURCES		
Adzuna	developer.adzuna.com	**FREE**
Jooble	jooble.org/api (email for key)	**FREE**
RapidAPI (LinkedIn+Indeed)	rapidapi.com	**$0–$10/mo**
AI & ENRICHMENT		
Anthropic (Claude)	console.anthropic.com	**Pay per token**
Clearbit (logos+company)	clearbit.com/developers	**50 free/mo**
COMMUNICATIONS & INFRA		
SendGrid (email)	sendgrid.com	**100/day free**
Twilio (SMS OTP)	twilio.com	**$0.05 per SMS**
Algolia (search)	algolia.com	**10K queries free**

🚀 **HirQube — Built with dedication. Good luck, Mahesh!**

Keep this guide saved. Refer back to each day's section as you build. You've got this.